Purpose

GATE-0 — gates/ Purpose + boundaries

Piece A — Purpose (what gates/ is)

File: `Axion/libraries/gates/GATE-0_purpose.md`

id: GATE-0
library: gates
schema_version: 1.0.0
status: draft

GATE-0 — gates/ Purpose + Boundaries

Purpose
`gates/` defines the **Gate DSL and evaluation contract** for Axion:
- how pass/fail checks are expressed (predicates)
- what data gates are allowed to read
- how evidence is collected and reported
- how gate outcomes interact with run control (hard stop vs pause vs warn)

This is the single source of truth for gate semantics across the pipeline.

What it governs (in scope)
- Gate definition model (gate_id, severity, predicates, evidence)
- Gate DSL grammar (operators, functions, allowed forms)
- Evaluation runtime contract (inputs, outputs, evaluation trace)
- Gate report schema (failures, pointers, remediation)
- Deterministic evaluation rules (ordering, stable IDs, replay behavior)
- Policy hook points for overrides (the hook mechanism; policy meaning is elsewhere)

What it does NOT govern (out of scope)
- Pipeline stage order and stage IO contracts → `orchestration/`
- Risk class meanings, override rules, expiry policy → `policy/`
- Workspace/pins/adapters/quotas/notifications → `system/`
- Any domain-specific checks content (e.g., knowledge gates, template completeness rules) lives in their libraries; `gates/` only defines the DSL + reporting format.

Consumers
- Gate evaluator engine (runtime)
- Orchestrator (pause/stop decisions)
- Operator UI (gate failure viewer + trace mode)
- Verification system (proof artifacts referenced by gate evidence)

Piece B — Boundary checklist

File: `Axion/libraries/gates/GATE-0_boundary_checklist.md`

id: GATE-0a
library: gates
schema_version: 1.0.0
status: draft

GATE-0a — Boundary Checklist

Belongs in `gates/` if it answers:
- “How do we express a gate check?”
- “How does the evaluator interpret predicates?”
- “What evidence format is emitted for pass/fail?”
- “What data can predicates read?”

Does NOT belong in `gates/` if it answers:
- “What stages exist / order?” (orchestration)
- “What is the risk class / who can override?” (policy)
- “What specific template/knowledge/standards rules exist?” (those libraries define content; gates defines language/engine)

Say **“next”** for **GATE-1 Gate definition model (gate_id, severity, predicate, evidence refs)**.

Gate definition model

GATE-1 — Gate definition model (gate_id, severity, predicate, evidence)

Piece A — Gate model (concept)

File: `Axion/libraries/gates/GATE-1_gate_definition_model.md`

```
---
library: gates
id: GATE-1
schema_version: 1.0.0
status: draft
---

# GATE-1 — Gate Definition Model

## Purpose
Define the canonical structure of a gate:
- what it is checking (predicates)
- what happens when it fails (severity)
- what evidence must be produced (evidence requirements)
- how it can be overridden (hook point, not the policy meaning)

## Core entities
- **Gate**: identified by gate_id, contains ordered predicates and reporting rules.
- **Predicate**: an atomic boolean test expressed in Gate DSL.
- **Evidence requirement**: what must be attached when predicate fails (pointers, diffs, proof refs).
- **Severity**: how orchestration responds (hard stop, pause, warn).

## Minimal gate fields
- gate_id (stable)
- title
- severity (hard_stop | pause | warn)
- predicates[] (ordered)
- evidence_policy (what to include in reports)
- override_hook (optional: allows policy engine to approve/deny override)
```

Piece B — Gate schema (JSON Schema)

File: `Axion/libraries/gates/schemas/gate_definition.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "gate_definition.v1",
  "type": "object",
  "required": [
    "gate_id",
    "title",
    "severity",
    "predicates",
    "evidence_policy",
    "created_at",
    "updated_at",
    "owner"
  ],
  "properties": {
    "gate_id": { "type": "string", "pattern": "^G\\d+_[A-Z0-9_]+$" },
    "title": { "type": "string", "minLength": 3 },

    "severity": { "type": "string", "enum": ["hard_stop", "pause", "warn"] },

    "predicates": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/predicate" }
    },

    "evidence_policy": {
      "type": "object",
      "required": ["mode"],
      "properties": {
        "mode": { "type": "string", "enum": ["minimal", "standard", "trace"] },
        "include_pointers": { "type": "boolean", "default": true },
        "include_diff": { "type": "boolean", "default": false },
        "include_verification_proof": { "type": "boolean", "default": true }
      },
      "additionalProperties": false
    },

    "override_hook": {
      "type": "object",
      "description": "If present, failing gate may request policy approval to override.",
```

```
      "required": ["hook_point"],
      "properties": {
        "hook_point": { "type": "string", "enum": ["GATE_OVERRIDE"] },
        "requires_reason": { "type": "boolean", "default": true }
      },
      "additionalProperties": false
    },

    "created_at": { "type": "string", "format": "date-time" },
    "updated_at": { "type": "string", "format": "date-time" },
    "owner": { "type": "string", "minLength": 2 }
  },

  "$defs": {
    "predicate": {
      "type": "object",
      "required": ["predicate_id", "expr", "message_on_fail"],
      "properties": {
        "predicate_id": { "type": "string", "pattern": "^PRED-[A-Z0-9]{4,}$" },
        "expr": { "type": "string", "minLength": 1, "description": "Gate DSL expression string" },
        "message_on_fail": { "type": "string", "minLength": 3 },
        "evidence_tags": {
          "type": "array",
          "items": { "type": "string" },
          "description": "Hints for what evidence to attach (paths, diffs, proofs)"
        }
      },
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Gate registry schema (multiple gates)

File: `Axion/libraries/gates/schemas/gate_registry.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "gate_registry.v1",
```

```
  "type": "object",
  "required": ["registry_id", "schema_version", "gates"],
  "properties": {
    "registry_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "gates": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "gate_definition.v1.schema.json" }
    }
  },
  "additionalProperties": false
}
```

Piece D — Determinism rules (gate evaluation order + identity)

File: `Axion/libraries/gates/GATE-1_determinism_rules.md`

```
---
library: gates
id: GATE-1a
schema_version: 1.0.0
status: draft
---

# GATE-1a — Determinism Rules (Gate Definitions)

- gate_id is immutable.
- predicates are evaluated in listed order.
- predicate_id is immutable within a gate definition.
- evidence_policy controls report verbosity but not pass/fail outcome.
- override_hook only requests approval; it does not change evaluation semantics.
```

Piece E — Validation checklist

File: `Axion/libraries/gates/GATE-1_validation_checklist.md`

```
---
library: gates
id: GATE-1b
```

schema_version: 1.0.0
status: draft

GATE-1b — Validation Checklist

- [ ] gate_definition validates against schema
- [ ] gate_id matches pattern and is unique (registry-level)
- [ ] each predicate has predicate_id, expr, message_on_fail
- [ ] predicates array order is preserved
- [ ] evidence_policy.mode is valid
- [ ] if override_hook present: hook_point == GATE_OVERRIDE

Gate DSL grammar

GATE-2 — Gate DSL grammar (operators, functions, allowed forms)

Piece A — DSL grammar spec (human-readable)

File: `Axion/libraries/gates/GATE-2_dsl_grammar.md`

---
library: gates
id: GATE-2
schema_version: 1.0.0
status: draft

GATE-2 — Gate DSL Grammar

Purpose
Define a minimal, deterministic expression language for gates.

Expression form
A predicate expr is a single expression that evaluates to boolean.

Supported forms:
- comparison: <value> == <value>, !=, >, >=, <, <=
- boolean: and, or, not
- function call: fn(arg1, arg2, ...)
- membership: contains(list, value)

No loops, no arbitrary code, no side effects.

Literals
- strings: "text"
- numbers: 123, 1.23
- booleans: true, false
- arrays: ["a","b"]

Paths (data access)
Paths reference the evaluation context only:
- selection.selected.length
- manifest.runtime.risk_class
- artifacts.has("CANONICAL_SPEC")
- ledger.complete

No filesystem reads inside DSL; evidence pointers are provided by the evaluator.

Operators
- ==, !=, >, >=, <, <=
- and, or, not

Standard functions (minimum)
- has_artifact(contract_id)
- coverage(category) # SPEC_INDEX coverage or artifact coverage
- maturity_at_least(level) # compares maturity enum
- targets_contains(name) # run target activation
- ledger_has(requirement) # proof ledger contains requirement
- allow_override(gate_id) # consults policy decision already recorded

---

Piece B — Machine-readable DSL function registry

File: `Axion/libraries/gates/registries/gate_dsl_functions.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "gate_dsl_functions.v1",
  "functions": [
    {
      "name": "has_artifact",
      "args": ["contract_id:string"],
      "returns": "boolean",
      "reads": ["run_manifest.artifacts"]
    },
    {
      "name": "coverage",
      "args": ["category:string"],
      "returns": "number",
      "reads": ["spec_index", "coverage_map"]
    },
    {
      "name": "maturity_at_least",
      "args": ["level:string"],
      "returns": "boolean",
      "reads": ["knowledge_selection.selected[].restrictions.maturity"]
    },
    {
```

```
    "name": "targets_contains",
    "args": ["target_name:string"],
    "returns": "boolean",
    "reads": ["run_manifest.runtime.profile_id", "canonical_spec.targets"]
  },
  {
    "name": "ledger_has",
    "args": ["requirement:string"],
    "returns": "boolean",
    "reads": ["proof_ledger"]
  },
  {
    "name": "allow_override",
    "args": ["gate_id:string"],
    "returns": "boolean",
    "reads": ["run_manifest.runtime.policy_decisions"]
  }
 ]
}
```

Piece C — Expression validation rules (parser constraints)

File: `Axion/libraries/gates/GATE-2_expression_validation_rules.md`

```
---
library: gates
id: GATE-2a
schema_version: 1.0.0
status: draft
---

# GATE-2a — Expression Validation Rules

A predicate expr is valid only if:
- It uses only supported operators: == != > >= < <= and or not
- It calls only functions in gate_dsl_functions registry
- It references only allowed context paths (no filesystem access)
- It contains no assignment, no imports, no arbitrary code syntax
- Max length: 300 chars (prevents hidden complexity)

Rejected examples:
- `exec("rm -rf /")`
```

- `open("file")`
- `while(...)`
- `import ...`

Piece D — Determinism rules (evaluation semantics)

File: `Axion/libraries/gates/GATE-2_determinism_rules.md`

```
---
library: gates
id: GATE-2b
schema_version: 1.0.0
status: draft
---

# GATE-2b — Determinism Rules (Gate DSL)

- Functions are pure (no side effects).
- Function results depend only on evaluation context.
- Arrays and objects used in evaluation must be pre-normalized (sorted where relevant).
- Short-circuit semantics apply:
  - A and B: if A is false, B not evaluated
  - A or B: if A is true, B not evaluated
- Numeric comparisons are deterministic (no floats from external sources unless normalized).
```

Piece E — Validation checklist

File: `Axion/libraries/gates/GATE-2_validation_checklist.md`

```
---
library: gates
id: GATE-2c
schema_version: 1.0.0
status: draft
---

# GATE-2c — Validation Checklist

- [ ] DSL functions registry exists and lists all callable functions
- [ ] Predicate expr strings use only allowed operators/syntax
```

- [ ] No disallowed tokens (exec/open/import/while/etc.)
- [ ] Expr length <= 300 chars
- [ ] Functions are pure and read only declared context paths

Gate evaluation runtime

GATE-3 — Gate evaluation runtime (inputs, outputs, trace)

Piece A — Evaluation runtime model (concept)

File: `Axion/libraries/gates/GATE-3_evaluation_runtime.md`

```
---
library: gates
id: GATE-3
schema_version: 1.0.0
status: draft
---

# GATE-3 — Gate Evaluation Runtime

## Purpose
Define the runtime contract for evaluating gates:
- required inputs (context)
- evaluation outputs (pass/fail + trace)
- evidence collection rules
- policy override integration (hook, not meaning)

## Inputs (evaluation context)
Minimum inputs to evaluate a gate:
- run manifest (pipeline/pins/runtime/artifacts)
- stage reports (for evidence pointers)
- selected artifacts (canonical spec, plan, envelopes, etc.)
- proof ledger (verification evidence)
- knowledge selection (if gate depends on it)
- policy decisions recorded in manifest (for allow_override)

## Outputs
Evaluation produces:
- gate_report (pass/fail + failures + evidence pointers)
- optional evaluation_trace (predicate-by-predicate execution trace)
```

Piece B — Evaluation request schema

File: `Axion/libraries/gates/schemas/gate_eval_request.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "gate_eval_request.v1",
  "type": "object",
  "required": ["request_id", "run_id", "gate_id", "context_refs", "requested_at"],
  "properties": {
    "request_id": { "type": "string", "pattern": "^GATEREQ-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string", "pattern": "^RUN-[A-Z0-9]{6,}$" },
    "gate_id": { "type": "string" },

    "context_refs": {
      "type": "object",
      "required": ["manifest_ref"],
      "properties": {
        "manifest_ref": { "type": "string", "minLength": 3 },
        "stage_reports": { "type": "array", "items": { "type": "string" } },
        "artifacts": { "type": "array", "items": { "type": "string" } },
        "proof_ledger_ref": { "type": "string" },
        "knowledge_selection_ref": { "type": "string" }
      },
      "additionalProperties": false
    },

    "trace_mode": { "type": "string", "enum": ["off", "on"], "default": "off" },

    "requested_at": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": false
}
```

Piece C — Evaluation trace schema (predicate-by-predicate)

File: `Axion/libraries/gates/schemas/gate_eval_trace.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "gate_eval_trace.v1",
  "type": "object",
```

```
  "required": ["run_id", "gate_id", "evaluated_at", "steps"],
  "properties": {
    "run_id": { "type": "string" },
    "gate_id": { "type": "string" },
    "evaluated_at": { "type": "string", "format": "date-time" },
    "steps": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["predicate_id", "expr", "result"],
        "properties": {
          "predicate_id": { "type": "string" },
          "expr": { "type": "string" },
          "result": { "type": "boolean" },
          "resolved_values": {
            "type": "object",
            "description": "Optional: extracted values used in evaluation (safe, small)",
            "additionalProperties": true
          },
          "evidence_refs": {
            "type": "array",
            "items": { "type": "string" },
            "description": "Pointers to evidence attached for this predicate"
          }
        },
        "additionalProperties": false
      }
    }
  },
  "additionalProperties": false
}
```

Piece D — Evidence collection rules

File: `Axion/libraries/gates/GATE-3_evidence_collection_rules.md`

```
---
library: gates
id: GATE-3a
schema_version: 1.0.0
status: draft
```

GATE-3a — Evidence Collection Rules

Purpose
Ensure gate failures are actionable without embedding large content blobs.

Evidence types (minimum)
- pointers: artifact paths + line ranges (if applicable)
- diffs: reference to diff artifact (optional)
- proof refs: references into proof ledger (verification commands)
- context snapshot refs: small safe fields extracted (counts, ids)

Evidence policy modes
- minimal: only pointers + failing predicate message
- standard: pointers + small context snapshot + proof refs
- trace: standard + full predicate-by-predicate trace

No content dumping
- Do not embed full documents, full logs, or long code blocks in gate reports.
- Evidence is references/pointers; content lives in artifacts.

Piece E — Determinism rules (runtime)

File: `Axion/libraries/gates/GATE-3_determinism_rules.md`

---
library: gates
id: GATE-3b
schema_version: 1.0.0
status: draft

GATE-3b — Determinism Rules (Evaluation Runtime)

- The evaluator reads only context_refs specified in the request.
- Gate definitions and DSL function registry must be pinned via SYS pins.
- Trace mode does not affect pass/fail results, only reporting.
- Evidence references must be stable paths (and hashes if available).
- Predicate evaluation order is the gate definition order.

Piece F — Validation checklist

File: `Axion/libraries/gates/GATE-3_validation_checklist.md`

---
library: gates
id: GATE-3c
schema_version: 1.0.0
status: draft

GATE-3c — Validation Checklist

- [ ] gate_eval_request validates against schema
- [ ] trace schema exists and is used only when trace_mode==on
- [ ] evaluator reads only declared context refs
- [ ] evidence modes (minimal/standard/trace) are supported
- [ ] pinned gate definition + function registry are required for reproducibility

Say **“next”** for **GATE-4 Gate report format (predicate failures, pointers, remediation)**.

Gate report schema

GATE-4 — Gate report format (pass/fail + failures + evidence)

Piece A — Gate report model (concept)

File: `Axion/libraries/gates/GATE-4_gate_report_model.md`

```
---
library: gates
id: GATE-4
schema_version: 1.0.0
status: draft
---

# GATE-4 — Gate Report Model

## Purpose
A gate report is the canonical output of evaluating a gate. It must answer:
- did the gate pass?
- which predicates failed?
- what evidence points to the problem?
- what remediation is recommended?
- was an override requested/granted?

## Design constraints
- Small, structured, machine-readable
- Evidence is pointers/refs, not full content
- Deterministic ordering for failures and evidence
```

Piece B — Gate report schema (JSON Schema)

File: `Axion/libraries/gates/schemas/gate_report.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "gate_report.v1",
  "type": "object",
  "required": [
    "run_id",
```

```
  "gate_id",
  "status",
  "evaluated_at",
  "failures",
  "evidence_policy",
  "summary"
],
"properties": {
  "run_id": { "type": "string" },
  "gate_id": { "type": "string" },

  "status": { "type": "string", "enum": ["pass", "fail", "warn"] },
  "evaluated_at": { "type": "string", "format": "date-time" },

  "summary": {
    "type": "object",
    "required": ["failed_predicates", "notes"],
    "properties": {
      "failed_predicates": { "type": "integer", "minimum": 0 },
      "notes": { "type": "array", "items": { "type": "string" } }
    },
    "additionalProperties": false
  },

  "evidence_policy": {
    "type": "object",
    "required": ["mode"],
    "properties": {
      "mode": { "type": "string", "enum": ["minimal", "standard", "trace"] }
    },
    "additionalProperties": false
  },

  "failures": {
    "type": "array",
    "items": { "$ref": "#/$defs/failure" }
  },

  "override": {
    "type": "object",
    "description": "Present when gate supports override or override was requested/granted",
    "properties": {
      "requested": { "type": "boolean" },
      "granted": { "type": "boolean" },
```

```
      "decision_id": { "type": "string" },
      "reason": { "type": "string" },
      "expires_at": { "type": "string", "format": "date-time" }
    },
    "additionalProperties": false
  },

  "trace_ref": {
    "type": "object",
    "description": "Present only when evidence_policy.mode == trace",
    "properties": {
      "path": { "type": "string" },
      "hash": { "type": "string" }
    },
    "additionalProperties": false
  }
},

"$defs": {
  "failure": {
    "type": "object",
    "required": ["predicate_id", "message", "evidence_refs", "remediation"],
    "properties": {
      "predicate_id": { "type": "string" },
      "message": { "type": "string", "minLength": 3 },

      "evidence_refs": {
        "type": "array",
        "items": { "$ref": "#/$defs/evidence_ref" }
      },

      "remediation": {
        "type": "array",
        "minItems": 1,
        "items": { "type": "string", "minLength": 3 }
      }
    },
    "additionalProperties": false
  },

  "evidence_ref": {
    "type": "object",
    "required": ["kind", "ref"],
    "properties": {
```

```
      "kind": { "type": "string", "enum": ["pointer", "diff", "proof", "snapshot"] },
      "ref": { "type": "string", "minLength": 3 },
      "notes": { "type": "string" }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece C — Determinism rules (report ordering + stability)

File: `Axion/libraries/gates/GATE-4_determinism_rules.md`

```
---
library: gates
id: GATE-4a
schema_version: 1.0.0
status: draft
---

# GATE-4a — Determinism Rules (Gate Reports)

- failures are ordered by gate predicate order.
- evidence_refs are ordered deterministically (kind order: pointer, snapshot, diff, proof; then ref lexicographic).
- remediation is short and actionable (no essays).
- trace_ref exists only in trace mode.
- status computation:
  - if any predicate fails and severity==hard_stop/pause → fail
  - if predicate fails and severity==warn → warn
```

Piece D — Minimal example gate report

File: `Axion/libraries/gates/templates/gate_report.example.json`

```
{
  "run_id": "RUN-ABC123",
  "gate_id": "G2_CANONICAL_INTEGRITY",
```

```
  "status": "fail",
  "evaluated_at": "2026-03-05T00:10:00Z",
  "summary": {
    "failed_predicates": 1,
    "notes": ["Canonical spec missing required relationships."]
  },
  "evidence_policy": { "mode": "standard" },
  "failures": [
    {
      "predicate_id": "PRED-0001",
      "message": "Canonical spec must include entity relationships.",
      "evidence_refs": [
        { "kind": "pointer", "ref":
"runs/RUN-ABC123/artifacts/CANONICAL_SPEC.json#relationships", "notes": "Field missing or
empty." }
      ],
      "remediation": [
        "Update canonical builder to populate relationships.",
        "Re-run S4_CANONICAL_BUILD and downstream stages."
      ]
    }
  ],
  "override": { "requested": false, "granted": false }
}
```

Piece E — Validation checklist

File: `Axion/libraries/gates/GATE-4_validation_checklist.md`

```
---
library: gates
id: GATE-4b
schema_version: 1.0.0
status: draft
---

# GATE-4b — Validation Checklist

- [ ] gate_report validates against schema
- [ ] failures are ordered by predicate order
- [ ] evidence_refs use only allowed kinds and stable refs
- [ ] trace_ref present only when evidence_policy.mode==trace
```

- [ ] override fields are coherent (requested/granted/decision_id/expiry)

Determinism rules

GATE-5 — Determinism + replay rules (stable evaluation + reproducibility)

Piece A — Determinism + replay contract

File: `Axion/libraries/gates/GATE-5_determinism_replay_rules.md`

```
---
library: gates
id: GATE-5
schema_version: 1.0.0
status: draft
---

# GATE-5 — Determinism + Replay Rules

## Purpose
Ensure gate outcomes are reproducible:
- same pinned inputs → same evaluation outcome
- evaluation can be replayed from recorded refs
- trace mode does not change pass/fail

## Replay prerequisites
A gate evaluation is replayable if the run manifest pins:
- gate registry version + path (and hash if used)
- DSL function registry version + path (and hash if used)
- all artifacts referenced by evaluation (paths + hashes if used)
- proof ledger referenced by evaluation

## Deterministic evaluation constraints
- Predicate evaluation order is fixed (gate definition order).
- Function calls are pure and read only the evaluation context.
- All collections used in evaluation must be normalized:
  - stable sort order for lists
  - stable key ordering for objects when serialized (if hashed)
- No wall-clock “now” inside DSL evaluation.
- No network access inside evaluation.

## Replay procedure (conceptual)
1) Load pinned gate definition for gate_id.
2) Load pinned function registry.
3) Load referenced artifacts/proof ledger by path (verify hashes if present).
```

4) Evaluate predicates in order.
5) Emit a gate_report with the same failure set (ordering may include same deterministic tie-breaks).

Allowed non-determinism (report-only)
These fields may differ between original and replay without violating determinism:
- evaluated_at timestamp
- trace_ref path if stored in a new location
All pass/fail and failure list content must remain identical.

Piece B — Replay request schema (optional but useful)

File: `Axion/libraries/gates/schemas/gate_replay_request.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "gate_replay_request.v1",
  "type": "object",
  "required": ["request_id", "run_id", "gate_id", "manifest_ref", "requested_at"],
  "properties": {
    "request_id": { "type": "string", "pattern": "^GATEREP-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },
    "gate_id": { "type": "string" },
    "manifest_ref": { "type": "string", "minLength": 3 },
    "requested_at": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": false
}
```

Piece C — Replay evidence requirements

File: `Axion/libraries/gates/GATE-5_replay_evidence_requirements.md`

---
library: gates
id: GATE-5a
schema_version: 1.0.0
status: draft

GATE-5a — Replay Evidence Requirements

A gate report should be considered replayable only if:
- run manifest includes pinned refs for:
  - gate registry
  - function registry
- every evidence_ref points to an artifact path that exists
- if hashing is enabled:
  - every referenced artifact has a hash recorded in manifest
  - replay validates hashes before evaluation

If any requirement is missing:
- replay mode must refuse OR mark output as "non-replayable" with reasons.

Piece D — Validation checklist

File: `Axion/libraries/gates/GATE-5_validation_checklist.md`

---
library: gates
id: GATE-5b
schema_version: 1.0.0
status: draft

GATE-5b — Validation Checklist

- [ ] Gate outcomes depend only on pinned definitions + referenced artifacts
- [ ] No wall-clock or network dependencies
- [ ] Replay prerequisites are defined and enforceable
- [ ] Trace mode changes only reporting fields, not pass/fail
- [ ] Replay request schema (if used) validates and references manifest

Minimum viable gate

GATE-6 — Minimum viable gates/ set (runtime-required files + “done”)

Piece A — MV gates inventory

File: `Axion/libraries/gates/GATE-6_minimum_viable_set.md`

---
library: gates
id: GATE-6
schema_version: 1.0.0
status: draft

GATE-6 — Minimum Viable gates/ Set

Goal
Define the smallest set of gates library assets required to evaluate gates deterministically.

Required runtime files (minimum)
1) Core docs
- GATE-0_purpose.md
- GATE-0_boundary_checklist.md
- GATE-1_gate_definition_model.md
- GATE-1_determinism_rules.md
- GATE-1_validation_checklist.md
- GATE-2_dsl_grammar.md
- GATE-2_expression_validation_rules.md
- GATE-2_determinism_rules.md
- GATE-2_validation_checklist.md
- GATE-3_evaluation_runtime.md
- GATE-3_evidence_collection_rules.md
- GATE-3_determinism_rules.md
- GATE-3_validation_checklist.md
- GATE-4_gate_report_model.md
- GATE-4_determinism_rules.md
- GATE-4_validation_checklist.md
- GATE-5_determinism_replay_rules.md
- GATE-5_replay_evidence_requirements.md
- GATE-5_validation_checklist.md
- GATE-6_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/gate_definition.v1.schema.json
- schemas/gate_registry.v1.schema.json
- schemas/gate_eval_request.v1.schema.json
- schemas/gate_eval_trace.v1.schema.json
- schemas/gate_report.v1.schema.json
- schemas/gate_replay_request.v1.schema.json (optional)

3) Registries (runtime defaults)
- registries/gate_dsl_functions.v1.json
- registries/gate_registry.axion.v1.json (your actual gate definitions; produced in this library)

4) Templates/examples (optional)
- templates/gate_report.example.json

Piece B — Definition of done (gates/)

File: `Axion/libraries/gates/GATE-6_definition_of_done.md`

---
library: gates
id: GATE-6a
schema_version: 1.0.0
status: draft

GATE-6a — gates/ Definition of Done

gates/ is “done” when:

Schemas + registries
- [ ] All gates schemas validate (JSON Schema check)
- [ ] gate_dsl_functions registry validates
- [ ] gate_registry validates and contains all required gate definitions for the pipeline

Runtime behavior (contract-level)
- [ ] Gate evaluator can parse + validate predicate expr strings
- [ ] Gate evaluator evaluates predicates deterministically from pinned context
- [ ] Gate evaluator emits gate_report with ordered failures + evidence refs
- [ ] Trace mode emits a trace artifact without changing pass/fail

Replay

- [ ] Replay rules are enforced: pinned gate registry + pinned function registry + referenced artifacts
- [ ] Replayed evaluations produce identical pass/fail and failure lists (timestamps may differ)

Piece C — Minimal folder tree

File: `Axion/libraries/gates/GATE-6_minimal_tree.txt`

```
Axion/libraries/gates/
  GATE-0_purpose.md
  GATE-0_boundary_checklist.md
  GATE-1_gate_definition_model.md
  GATE-1_determinism_rules.md
  GATE-1_validation_checklist.md
  GATE-2_dsl_grammar.md
  GATE-2_expression_validation_rules.md
  GATE-2_determinism_rules.md
  GATE-2_validation_checklist.md
  GATE-3_evaluation_runtime.md
  GATE-3_evidence_collection_rules.md
  GATE-3_determinism_rules.md
  GATE-3_validation_checklist.md
  GATE-4_gate_report_model.md
  GATE-4_determinism_rules.md
  GATE-4_validation_checklist.md
  GATE-5_determinism_replay_rules.md
  GATE-5_replay_evidence_requirements.md
  GATE-5_validation_checklist.md
  GATE-6_minimum_viable_set.md
  GATE-6_definition_of_done.md
  GATE-6_minimal_tree.txt

  schemas/
    gate_definition.v1.schema.json
    gate_registry.v1.schema.json
    gate_eval_request.v1.schema.json
    gate_eval_trace.v1.schema.json
    gate_report.v1.schema.json
    gate_replay_request.v1.schema.json

  registries/
    gate_dsl_functions.v1.json
```

```
  gate_registry.axion.v1.json

templates/
  gate_report.example.json
```

Say **“next”** to start the next library folder. Recommended next is **policy/** (POL-0).